

南京工程学院
NANJING INSTITUTE OF TECHNOLOGY

OULUN YLIOPISTO

Special Course in Software Engineering ‘26

Project – Integration and Simulation

Important

- Work in groups
  - Therefore, you must sit with your group members
  - Communicating over WeChat or similar application is not acceptable
  - If you don't like this, then you are free to leave, and you'll be marked absent

IMPORT and CLONE the following repository:

https://github.com/prabhatram/project_integration_and_simulation

These files must be added to your original project repository, and not saved as a brand new repository

Only the developer_agent.py file must be updated.

Everything else is new

Test Agent

- Create a test agent
- This agent is responsible for creating the tests based on the requirements brief, the plan created by the planner agent, and the `decide_next_move` function generated by the developer agent.
- This agent must create tests that tests for all the possible scenarios and makes sure the `decide_next_move` function works as expected.
- Just follow the instructions given in the `test_agent.py` file

The Orchestrator

- Right now, your different agents have to be run separately.
- Now, you must integrate these separate agents so that they all are run one after the other from a central location. This is the `main.py` file, and instructions are available there
- The file should do the following and record everything in a log file, saved as a `trace.log`
  - Load the brief
  - Run the Analyst agent
  - Run the Planner agent
  - Run the Developer agent
  - Runs the test agent
  - Runs the simulation
  - Runs the reviewer agent
  - Runs the IRSIM benchmark
  - Runs the benchmark reviewer
- All of the above runs must be logged

Simulation Runner

- Create a simple simulation to test the navigation logic (**`decide_next_move`**) generated by the developer agent.
- In this simulation, you test if the logic inside the **`decide_next_move`** produces the expected output when tested against all the possible scenarios.
- You also store the result of the entire simulation in a **`JSON`** file called the **`simulation_report.json`**
- Follow the instructions in the **`runner.py`**

Reviewer Agent

- The reviewer agent will review the simulation report generated by the **`runner.py`**
- This agent will identify problems based on the report and recommend improvements
- Follow the instructions given in **`reviewer_agent.py`**

IR-SIM

- IR-SIM provides a realistic environment where you can visually confirm if the navigation logic generated by the developer agent indeed works
- First install the library: **`uv pip install ir-sim`**
- The folder irsim_worlds has four simulated environments of different difficulty level: easy, medium, difficult, and custom
- These worlds must be simulated and tested one by one by running the following, one after another:

```
./run_visual.sh irsim_worlds/world_easy.yaml
./run_visual.sh irsim_worlds/world_medium.yaml
./run_visual.sh irsim_worlds/world_difficult.yaml
./run_visual.sh irsim_worlds/world_custom.yaml
```

The file **`run_visual.sh`** and the file it needs, **`test_irsim_with_agent.py`**, are provided. You don't need to change anything in these files.

If your navigation logic is correct, then each of the above runs will show you a visual where the robot will move to its destination, avoiding obstacles.

Automated Benchmarking

- In addition to visual inspection, you also need to record and measure performance of your robot (and the navigation logic) based on some metrics.
- Load all the IR-SIM world files, one by one, by calling the **`test_irsim_with_agent.py`**.
- Create a report based on each world's run, capturing different metrics, as instructed in the **`benchmark_irsim.py`** file
  - Success Rate: Goals reached / Total environments
  - Efficiency rate: Efficient runs / Total environments
  - Average steps: Total steps / Total environments

Benchmark Reviewer

- You will create another program that reviews the benchmark result.
  - This is similar to the program **`reivewer_agent.py`** that reviewed the simulation result.
- The program measures quality requirements like safety, goal completion, and efficiency
- The program should produce a report that tells the result of the review of the benchmark report, and recommends further actions, like if revision of the code is needed.
  - The program should also specify precisely what issue it found in the report and recommend how that can be improved/fixed
- Follow the instructions given in **`reviewer_benchmark_agent.py`**

Revised navigation logic

- Here, you need to update your original developer_agent.py file.
- Create a new function like **run_developer_revision()**
  - This function will receive the original navigation code, the review notes, the benchmark report, the requirements and the plan.
  - Based on all of the above, this function will ask the LLM to generate a new navigation logic and then save that as a new file
- Re-test the new navigation logic and then re-run the entire pipeline, including the IR-SIM simulation to confirm everything works.

That’s all for this practice